LizardsVSCats: Analysis

COP4634
Project 3
Fall 2026
Team: Sarah Pierpoint, Melanie Vogt

Problem Description and how our code solves it:

A herd of lizards lives in a sago palm and finds food in the monkey grass across a driveway. Two cats sleep most of the time and wake up periodically to look at the driveway. If a cat sees more than four lizards crossing at once, it plays with them, and the program ends. The goal is to synchronize the lizard threads so that no more than four lizards are crossing at once, while still letting as many as possible cross at the same time, and without using busy waits.

Our solution uses two synchronization tools. A counting semaphore, initialized to 4, acts as a pool of driveway slots: a lizard takes a slot before crossing and returns it afterward, so a fifth lizard blocks until a permit becomes avaliable until one is free. A mutex protects the shared crossing counters that the lizards update and the cats read, preventing race conditions.

All changes made to the code

Cat threads and main() setup/cleanup

Cat::run() creates a new thread that executes catThread, and Cat::wait() calls join on that thread so the caller blocks until the cat finishes. A destructor deletes the thread object, and the constructor sets the thread pointer to NULL, so cat objects do not leak memory.

In main(), the semaphore and the mutex are initialized before any thread starts, because threads use them immediately. Main then creates the cat objects and starts the cat and lizard threads. After sleeping for WORLDEND seconds, main() acquires counterLock, sets running = 0, and releases the mutex to signal the thread to terminate.. The semaphore and mutex are destroyed, and the objects are deleted only after all joins complete, because destroying them while a thread could still use them is undefined behavior. A lizard destructor was also added so the lizard thread objects are freed.

Driveway Synchronization in the lizard functions

The synchronization functions that control when lizards can enter and leave the driveway have been completed. Both sago2MonkeyGrassIsSafe() and monkeyGrass2SagoIsSafe() use sem_wait(&driveway) to acquire a permit before a lizard begins crossing. The semaphore is initialized to MAX_LIZARD_CROSSING, which is set to four. If all four permits are in use, another lizard must wait until a permit becomes available.

After a lizard finishes crossing, madeIt2MonkeyGrass() or madeIt2Sago() calls sem_post(&driveway) to release its permit. This allows another waiting lizard to cross. Both directions use the same semaphore so that the total number of lizards crossing cannot exceed four, regardless of their direction. Lizard::lizardThread() was also completed by adding calls to the existing functions in the order specified by the assignment. Each lizard sleeps, waits for permission to cross to the monkey grass, crosses, releases its permit, eats, waits for permission to return, crosses back to the sago palm, and releases its permit again. The sequence repeats while the program simulation is running.

To prevent a race condition involving the shared running variable, an update to the lizard and cat thread loops to check its value while holding counterLock was made. The original while(running) loops were changed to while(true) loops with a protected check and a break statement when the simulation ends. The shutdown operation in main() also uses the same mutex when setting running = 0. These changes ensure that concurrent reads and writes of the shared run-state variable are synchronized.

Crossing functions and counter protection

In crossSago2MonkeyGrass() and crossMonkeyGrass2Sago(), the mutex is held while the crossing counter is incremented and again when it is decremented, but it is released during the sleep that simulates the crossing. Holding the lock during the sleep would let only one lizard cross at a time, which would violate the requirement to allow the maximum possible number of lizards. A maxCrossing variable, updated under the lock, records the highest simultaneous crossing count for the results table.

The cats also read these counters, so their check in catThread() is wrapped in the same mutex. Without it, counter++ could be interleaved by two threads, and an update lost, which is a race condition, and a cat could see an inconsistent total.

Makefile and Compilation

The Makefile compiles lizard.cpp with g++ using these flags

- -g: includes debugging symbols
- -Wall: enables all common warnings
- -std=c++11: needed for std::thread
- -pthread: links the POSIX threads library, which the threads, mutex, and semaphore all depend on

It also includes an all rule and a clean rule that removes the executable. The program compiles with no errors and no warnings. We also ran it under Valgrind, which reported no memory errors and no leaks.

Table 1: Results table

WORLDEND	Number of Lizards	Number of Cats	Maximum Number of Lizards Crossing	Lizards safe?
30	20	2	4	Yes
30	5	2	4	Yes
30	30	2	4	Yes
30	20	3	4	Yes
60	20	1	4	Yes
180	20	2	4	Yes

Experimental Results

The program was tested on the Linux department server using different simulation durations and numbers of lizards and cat threads. All six experiments exited successfully with a status code of 0. The maximum number of simultaneous crossings was four in every experiment, demonstrating that the driveway's full capacity could be used without an observed capacity violation.

The number of lizards was increased to 30 to test the program under greater contention and increased the number of cats to three to test additional concurrent access to the shared crossing counters. Both configurations completed successfully. Finally, the required configuration of 20 lizards, two cats, and WORLDEND set to 180 seconds. This test was also successful with a maximum of four lizards crossing simultaneously. No deadlocks or cat-triggered terminations were observed in the completed test runs.

Issues Encountered:

- Memory leak: Lizard objects never freed their thread objects. Adding a destructor fixed it, and Valgrind confirmed it.
- Interleaved debug output: with many threads printing at once, the -d output sometimes appeared scrambled because cout is not synchronized. It did not affect the counters or the correctness of the program.
- Race condition involving the running variable: during integration we noticed the main() modified the shared running variable while the lizard and cat threads read it without synchronization. It has been corrected using counterLock. Both thread loops now use a mutex-protected check to determine when they should terminate.